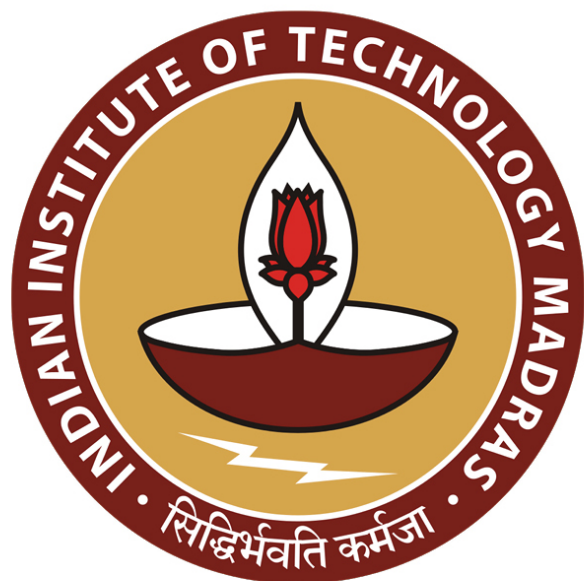


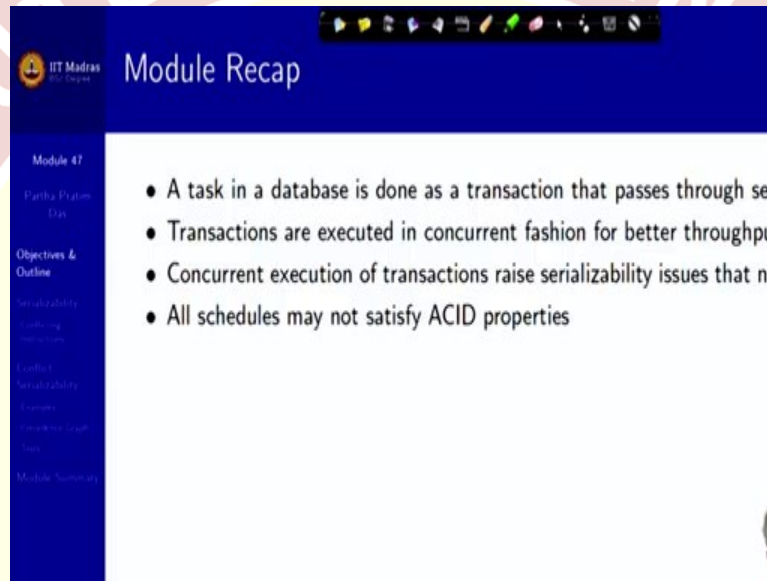


IIT Madras
ONLINE DEGREE

Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Transactions/2: Serializability

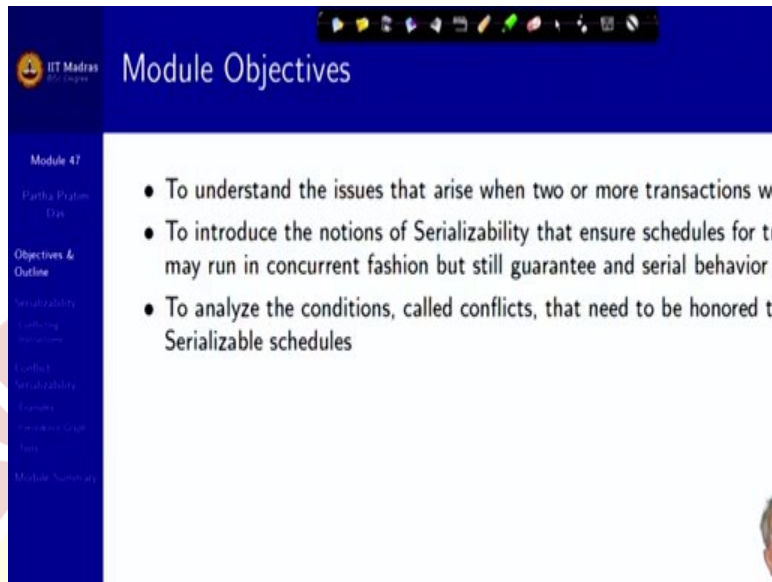
Welcome to Module 47 of Database Management Systems course in the IIT Madras online B.Sc. program.

(Refer Slide Time: 0:30)



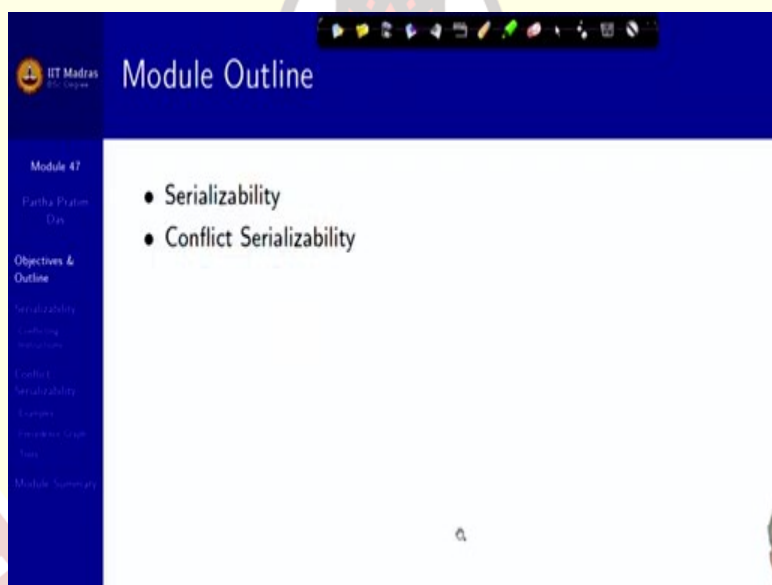
We have started discussing about transactions that is doing some changes work in the database. We have defined transactions, the states and the basic concept of concurrency where more than one transaction could operate on the same data items at the same time. And we have with that seen that a certain schedule of these instructions coming from different transactions seem to be valid and certain are not, some are serializable, some are not serializable, and all schedules may not satisfy the basic ACID properties of transactions that is atomicity, consistency, isolation and durability.

(Refer Slide Time: 1:30)



The slide is titled "Module Objectives" and features the IIT Madras logo in the top left corner. A vertical sidebar on the left lists the following items: "Module 47", "Partha Pratim Das", "Objectives & Outline", "Serializability", "Conflicting Transactions", "Conflict Serializability", "Examples", "Precedence Graph", "Deadlocks", and "Module Summary". The main content area contains three bullet points:

- To understand the issues that arise when two or more transactions w
- To introduce the notions of Serializability that ensure schedules for tr may run in concurrent fashion but still guarantee and serial behavior
- To analyze the conditions, called conflicts, that need to be honored to Serializable schedules

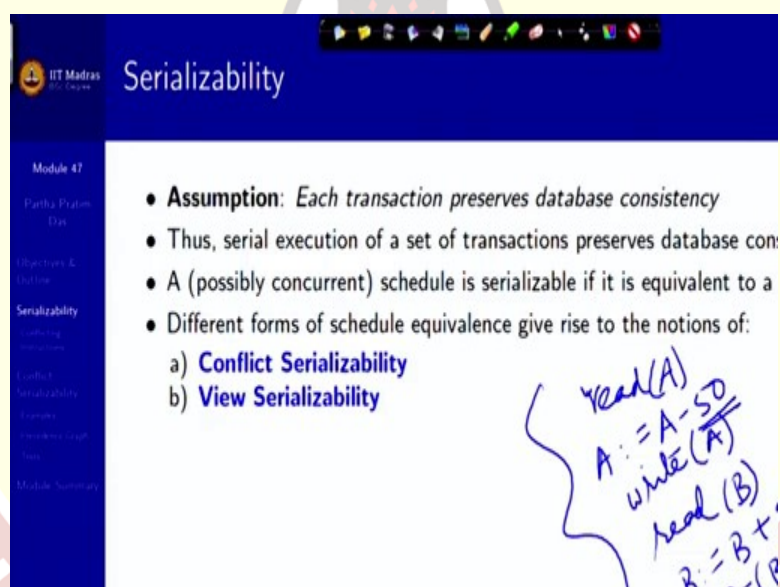


The slide is titled "Module Outline" and features the IIT Madras logo in the top left corner. A vertical sidebar on the left lists the following items: "Module 47", "Partha Pratim Das", "Objectives & Outline", "Serializability", "Conflicting Transactions", "Conflict Serializability", "Examples", "Precedence Graph", "Deadlocks", and "Module Summary". The main content area contains two bullet points:

- Serializability
- Conflict Serializability

So, moving on we will now take a closer look in terms of how transactions can work concurrently, what is the serializable behavior, what are the conflicts and so on and talk about serializability and conflict serializability.

(Refer Slide Time: 1:49)



Serializability

Module 47
Partha Pratim Das
Objectives & Outline
Serializability
Conflicts
Conflict Serializability
Examples
Two-Phase Locking
Two-Phase
Module Summary

- **Assumption:** *Each transaction preserves database consistency*
- Thus, serial execution of a set of transactions preserves database consistency
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule
- Different forms of schedule equivalence give rise to the notions of:
 - a) **Conflict Serializability**
 - b) **View Serializability**

Serializability is, we start with the assumption that each transactions preserves the database consistency. This is a basic assumption. If a transaction by itself has a logical error like if we have read A, then we have A assigned $A-50$, write A, then we have read B, B assigned $B+40$ and write B. This is a valid transaction, but I have deduced, reduced A by 50 and increased B by 40. So, $A+B$ here will be less by 10. That has got nothing to do with concurrence or anything. This inconsistency is due to a fallacious transaction. So, we are assuming that such transactions are not there.

So, the a serial execution of the set of transactions will always preserve the consistency, because if every transaction is preserving consistency, then if I do with them one after the other they will preserve consistency. A possibly concurrent schedule which mixes the instructions from multiple transactions is said to be serializable if it is equivalent to a serial schedule. If it produces exactly the same data values as some serial schedule if that can happen, then the concurrent schedule is called serializable. And there are two notions for serializability, one is conflict serializability and others view serializability.

(Refer Slide Time: 3:46)

Recap Schedule 3: Serializable

- Let T_1 and T_2 be the transactions defined previously. The following is a serial schedule, but it is **equivalent** to Schedule 1

Schedule 3		Schedule 1	
T_1	T_2	T_1	T_2
read (A)		read (A)	
$A := A - 50$		$A := A - 50$	
write (A)		write (A)	
	read (A)	read (B)	
	$temp := A * 0.1$	$B := B + 50$	
	$A := A - temp$	write (B)	
	write (A)	commit	
			read (A)
			$temp := A * 0.1$
			$A := A - temp$
			write (A)
read (B)			read (B)
$B := B + 50$			$B := B + temp$
write (B)			
commit			
	read (B)		
	$B := B + temp$		

A	B	A+B	Transact
100	200	300	@ Start
50	200	250	T1, v
45	200	245	T2, v
45	250	295	T1, v
45	255	300	T2, v

Consistent
Inconsistent
Inconsistent

So, just to recap, these examples, you should look back that we had shown in the last module that there is a schedule 3 which is serializable to schedule 1, because it is equivalent to schedule 1.

(Refer Slide Time: 4:02)

Recap Schedule 4: Not Serializable

- The following concurrent schedule does not preserve the sum of "A + B"

T_1	T_2
read (A)	
$A := A - 50$	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	$B := B + temp$

A	B	A+B	Transact
100	200	300	@ Start
90	200	290	T2, write A
50	200	250	T1, write A
50	250	300	T1, write B
50	255	305	T2, write B

Consistent @ Cor
Inconsistent @ Tr
Inconsistent @ Co

So, as we had seen that schedule 1, 2 and 3 are serializable, we also saw this schedule 4, which is not serializable because it does not preserve the sum of A+B because the A written here is overwritten here and the B written here is overwritten here resulting in a total sum, which is actually just 260, 40 less, so we concluded that all concurrent schedules are not serializable. So, these are just examples you had.

(Refer Slide Time: 4:35)

IIT Madras
Simplified View of Transactions

Module 47
Partha Pratim Das
Operations & Dataflow
Serializability
Locking
Concurrent Execution
Consistency
Deadlocks
Module Summary

- We ignore operations other than **read** and **write** instructions
 - Other operations happen in memory (are temporary in nature) and affect the state of the database
 - This is a simplifying assumption for analysis
- We assume that transactions may perform arbitrary computations on buffers in between reads and writes
- Our simplified schedules consist of only **read** and **write** instructions

So, now to talk about serializability let us make a simplifying assumption. See in the database, in terms of database consistency, what is, what are important is what you read and what you write. Everything else that you do with the data as to after reading what computations you do and what you want to write in between whatever competition you do is not a matter of the order in which you do things because that is a matter of logic, that is a matter of pure and simple logic, and the read write is what would affect the consistency of the database, unless, as I said, unless there is a logical error in a transaction. That is a different thing. So, we will assume that schedules consist only of read and write. Do not look at the rest.

(Refer Slide Time: 5:30)

Conflicting Instructions

Module 47
Partha Pratim Das
Objectives & Outline
Serializability
Conflicting Instructions
Conflict Serializability
Formal
Consistency and
Two-Phase
Module Summary

- Let l_i and l_j be two Instructions from transactions T_i and T_j respectively
- Instructions l_i and l_j conflict if and only if there exists some item Q and l_i , and at least one of these instructions write to Q
- a) $l_i = \text{read}(Q)$, $l_j = \text{read}(Q)$. l_i and l_j don't conflict
- b) $l_i = \text{read}(Q)$, $l_j = \text{write}(Q)$. They conflict
- c) $l_i = \text{write}(Q)$, $l_j = \text{read}(Q)$. They conflict
- d) $l_i = \text{write}(Q)$, $l_j = \text{write}(Q)$. They conflict
- Intuitively, a conflict between l_i and l_j forces a (logical) temporal order
 - If l_i and l_j are consecutive in a schedule and they do not conflict, remain the same even if they had been interchanged in the schedule

T_i
 T_j
 $T_i - T_j$

Conflicting Instructions

Module 47
Partha Pratim Das
Objectives & Outline
Serializability
Conflicting Instructions
Conflict Serializability
Formal
Consistency and
Two-Phase
Module Summary

- Let l_i and l_j be two Instructions from transactions T_i and T_j respectively
- Instructions l_i and l_j conflict if and only if there exists some item Q and l_j , and at least one of these instructions write to Q
- a) $l_i = \text{read}(Q)$, $l_j = \text{read}(Q)$. l_i and l_j don't conflict
- b) $l_i = \text{read}(Q)$, $l_j = \text{write}(Q)$. They conflict
- c) $l_i = \text{write}(Q)$, $l_j = \text{read}(Q)$. They conflict
- d) $l_i = \text{write}(Q)$, $l_j = \text{write}(Q)$. They conflict
- Intuitively, a conflict between l_i and l_j forces a (logical) temporal order
 - If l_i and l_j are consecutive in a schedule and they do not conflict, remain the same even if they had been interchanged in the schedule

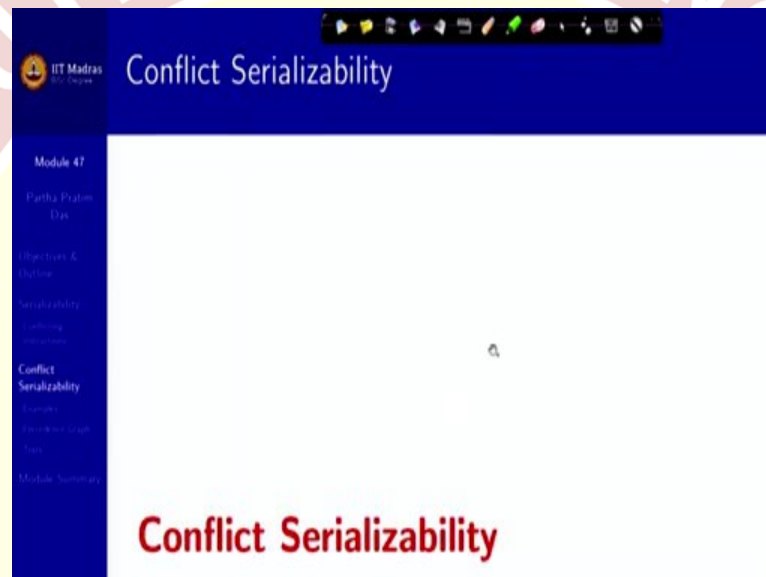
Now, we say suppose there are two transactions T_i and T_j , two instructions l_i and l_j and from transaction T_i and T_j . So, T_i is a transaction, T_j is a transaction, l_i is some instruction there, l_j is some instruction this. This is a situation. Now, we will say that there if for some data item Q , which is accessed by both these instructions l_i as well as l_j we will say that there is a conflict if at least one of them write to Q .

So, we say the first one both are reading. So, does it matter in which order they read, it does not. If I just looking at these two. So, they do not conflict. But if any of these, if one reads, the other write, one writes, the other reads, if both write, then it does matter who goes first and who goes

second. So, they conflict. So, any two instructions from two transactions if any one of them is writing which a data item which the other is reading or writing, then there is a conflict between these two instructions. And so in a serial schedule T_i, T_j or T_j, T_i , I will have either I_i going first before I_j or I_j going first before I_i , but I cannot mix them without having this conflict.

Now, at this point, it will be good to note that if I_i and I_j are consecutive in a schedule, then they do not conflict, because their result will remain the same even if they are interchanged.

(Refer Slide Time: 8:08)



So, let us talk about conflict serializability.

(Refer Slide Time: 8:12)

Conflict Serializability

- If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**.
- We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule.

We say we have a schedule S and we want to transform that to a schedule S prime. And what is the process of this transformation, doing a series of swaps of non-conflicting instructions. If there are non-conflicting instructions, we can swap them, change their temporal order. So, in that process from S if we produce S prime, then we say that S and S prime are conflict equivalent.

Now, we will say that schedule S is conflict serializable if it is conflict equivalent to a serial schedule. Serial schedule is one where first one transaction then the next transaction. This is a basic concept. So, we will have to see what is conflict serializable, and what is not.

(Refer Slide Time: 9:06)

Conflict Serializability (2)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where by a series of swaps of non-conflicting instructions:

- Swap $T_1.read(B)$ and $T_2.write(A)$
- Swap $T_1.read(B)$ and $T_2.read(A)$
- Swap $T_1.write(B)$ and $T_2.write(A)$
- Swap $T_1.write(B)$ and $T_2.read(A)$

These swaps work with (A or B) in diff

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(A)

T_1	T_2
read(A)	
write(A)	
	read(A)
read(B)	write(A)

T_1
read(A)
write(A)
read(B)
write(B)



Module 47
Parallel Program
Data
Operations &
Database
Serializability
Locking
Serializability
Conflict
Serializability
Consistency
Deadlocks
Two-Phase
Module Summary

Conflict Serializability (2)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where by a series of swaps of non-conflicting instructions:

- Swap $T_1.read(B)$ and $T_2.write(A)$ ✓
- Swap $T_1.read(B)$ and $T_2.read(A)$ ✓
- Swap $T_1.write(B)$ and $T_2.write(A)$ ✓
- Swap $T_1.write(B)$ and $T_2.read(A)$ ✓

These swaps work with (A or B) in diff

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(A)

T_1	T_2
read(A)	
write(A)	
	read(A)
read(B)	
	write(A)

T_1
read(A)
write(A)
read(B)
write(B)

Let us take an example from the earlier one itself that we have a schedule given here which does read write A, then T_2 does read write A, read write B, read write B this is. So, I can see, we can see that this write, read in T_1 and this write in T_2 are not conflicting. Why, because they are writing to, reading and writing two different read items, which means I can swap these two. Swapping these two means this goes up, this comes down.

Once I have done that, then I look at these two. These are also not conflicting, because as it is both of them are read and they are from different locations so I can swap that. Then I can look at write B, write A, two different locations I can swap them, write B can also move up. Then I can look at write B and read A, different locations that can also move up. If it does after these swaps, this is what I get. In every case I swapped non-conflicting instructions only.

Now, what is this? Here all instructions of one transaction is first T_1 and then T_2 . So, this is a serial schedule. So, which says that tells me that schedule 3 here shown is a conflict serializable schedule that is by swapping non-conflicting instructions I am able to convert scheduled 3 to a serial schedule, schedule 6. So, that is a conflict serializability.

(Refer Slide Time: 11:25)

IIT Madras
M.Tech Degree

Module 47

Partha Pratim Das

Objectives & Outline

Serializability

Locking

Conflicts

Conflict Serializability

Examples

Consistency & Deadlocks

Module Summary

Conflict Serializability (3)

- Example of a schedule that is not conflict serializable:

T ₃	T ₄
read (Q)	
write (Q)	write (Q)

• We are unable to swap instructions in the above schedule to obtain a serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$

Now, obviously, there are several schedules which are not conflict serializable, look at here read A, read Q, write Q and this does write Q. You cannot move anything. This is the direct conflict here. In fact, you are unable to swap instructions to either get a serial schedule T3, T4 or a serial schedule T4, T3. So, this is not conflict serializability.

(Refer Slide Time: 12:05)

IIT Madras
M.Tech Degree

Module 47

Partha Pratim Das

Objectives & Outline

Serializability

Locking

Conflicts

Conflict Serializability

Examples

Consistency & Deadlocks

Module Summary

Example: Bad Schedule

Consider two transactions:

Transaction 1

UPDATE accounts

SET balance = balance - 100

WHERE acct_id = 31414

Transaction 2

UPDATE accounts

SET balance = balance * 1.005

- In terms of read / write we can write these as:
 - Transaction 1: $r_1(A), w_1(A)$ // A is the balance for acct_id = 31414 ✓
 - Transaction 2: $r_2(A), w_2(A), r_2(B), w_2(B)$ // B is balance of other accounts ✓
- Consider schedule S:
 - Schedule S: $r_1(A) \ r_2(A) \ w_1(A) \ w_2(A) \ r_2(B) \ w_2(B)$

Example: Bad Schedule

Module 47
Partha Pratim Das
Objectives & Outline
Serializability
Locking
Concurrency
Consistent Serializability
Examples
Predefined Locking Times
Module Summary

Consider two transactions:

Transaction 1
UPDATE accounts ✓
SET balance = balance - 100
WHERE acct_id = 31414

Transaction 2
UPDATE accounts
SET balance = balance * 1.005

(i)
P₁
P₂
W₁
W₂
P₁
W₂

- In terms of read / write we can write these as:
 Transaction 1: $r_1(A), w_1(A)$ // A is the balance for acct_id = 31414
 Transaction 2: $r_2(A), w_2(A), r_2(B), w_2(B)$ // B is balance of other accounts
- Consider schedule S:
 Schedule S: $r_1(A) \quad r_2(A) \quad w_1(A) \quad w_2(A) \quad r_1(R) \quad w_2(R)$

Now, let us say how some schedules are bad and some schedules are good. Let us take two transactions accounts only, but little different. Now, update accounts balance is balance minus 100 for account ID. So, this is, this clearly says that 31414 is withdrawing 100 rupees from her account. This is transaction 2, which is crediting every account with 0.5 percent interest. So, there is no where clause, so it will happen for everybody balance is balance 1.005 which means by 0.5 percent yes it is given to everyone.

So, this is a transaction T1, this is transaction T2, read A, write A, this is, this will happen for all accounts. So, let us say there is, other than A there is another account B, there could be several others, but just for illustration we assume that there is another account B. So, it will have read A write A in transaction T1 for this. It will have read A, write A for transaction 2. It will have read B write B for transaction 2 again.

Now, if I do a, if we do a schedule like this read A in transaction 1, then read A in transaction 2, write A transaction 1, transaction 2 then that, will that give me a is this schedule serializable. So, suppose we have this 100 rupees in A and 200 rupees in A and 100 rupees in B. Now, if I do serially, if I do this schedule then what will happen? I am doing $w_1(A)$ that is write in A based on what I have read, which is 200 after doing a reduction of withdrawal of 100 rupees, so this will be 100 rupees.

But what I am writing here by w_2 , which I am doing next is based on what I read in A and given the interest, read in A was 200 given an interest of 0.5 percent that is 1 rupees. So, w_2 will write

201, B will have. So, you can see that this is not, why is it not serializable, because this is causing disaster. I have taken out 100 rupees from the account and then i end up having 201 rupees. So, the bank is going to get bankrupt very easily. So, this is S you can easily see is not a serializable schedule.

(Refer Slide Time: 15:13)

Example: Bad Schedule (2)

Module 47

Partha Pratim Das

Objectives & Outline

Serializability

Locking

Concurrent Serializability

Examples

Concurrency Control

Index

Module Summary

- Ideal schedule is serial:

Serial schedule 1:
 $r_1(A), w_1(A), r_2(A), w_2(A), r_2(B), w_2(B)$

Serial schedule 2:
 $r_2(A), w_2(A), r_2(B), w_2(B), r_1(A), w_1(A)$
- We call a schedule **serializable** if it has the same effect as some serial schedule regardless of the specific information in the database.
- As an example, consider Schedule T, which has swapped the third and fourth operations from S:

○ Schedule S : $r_1(A), r_2(A), w_1(A), w_2(A), r_2(B), w_2(B)$
 ○ Schedule T : $r_1(A), r_2(A), w_2(A), w_1(A), r_2(B), w_2(B)$

T1	Schedule
T2	A
Initial Value	200.00
Final Value	100.00
Initial Value	100.00
Final Value	100.50

A is \$100 initially

	A	B
(initial:)	100.00	100.
$r_1(A)$:		
$r_2(A)$:		
$w_2(A)$:	100.50	
$w_1(A)$:	0.00	
$r_2(B)$:		
$w_2(B)$:		100.

Srl

Example: Bad Schedule (2)

Module 47

Partha Pratim Das

Objectives & Outline

Serializability

Locking

Concurrent Serializability

Examples

Concurrency Control

Index

Module Summary

- Ideal schedule is serial:

Serial schedule 1:
 $r_1(A), w_1(A), r_2(A), w_2(A), r_2(B), w_2(B)$

Serial schedule 2:
 $r_2(A), w_2(A), r_2(B), w_2(B), r_1(A), w_1(A)$
- We call a schedule **serializable** if it has the same effect as some serial schedule regardless of the specific information in the database.
- As an example, consider Schedule T, which has swapped the third and fourth operations from S:

○ Schedule S : $r_1(A), r_2(A), w_1(A), w_2(A), r_2(B), w_2(B)$
 ○ Schedule T : $r_1(A), r_2(A), w_2(A), w_1(A), r_2(B), w_2(B)$

T1	Schedule
T2	A
Initial Value	200.00
Final Value	100.00
Initial Value	100.00
Final Value	100.50

A is \$100 initially

	A	B
(initial:)	100.00	100.
$r_1(A)$:		
$r_2(A)$:		
$w_2(A)$:	100.50	
$w_1(A)$:	0.00	
$r_2(B)$:		
$w_2(B)$:		100.

Srl

So, what are the ideal schedules? Ideal schedules would be one after the other first T1 then T2 or first T2 then T1. So, we have to see anything which is serializable has to be equivalent to one of them. And we saw S is not. We can try out some T where what we have done from S the order of

w1, w2 we have changed in this and you can see the effect here. You can again see that B is okay, but A is getting less now.

A should, after taking out 100 rupees it should get an interest of 0.5 percent. So, the valid values for A are 100.50 if you first withdraw and then give interest or it is first give interest so then it is 101 and then withdraw so it is 101. So, these two, by the serial schedule these are the two possibilities, but here you are ending up with different other values. So, T where you have changed, another schedule T where you have changed this is also not serializable. So, neither S nor T are serializable. These are bad schedules.

(Refer Slide Time: 16:59)

Example: Good Schedule

- What's a non-serial example of a serializable schedule?
 - We could credit interest to A first, then withdraw the money, then B:
 - Schedule U : $r_2(A), w_2(A), r_1(A), w_1(A), r_2(B), w_2(B)$
 - Initial: $A = 200, B = 100$
 - Final: $A = 101, B = 100.50$
- Schedule U is conflict serializable to Schedule 2:

Schedule U:	$r_2(A), w_2(A), r_1(A), w_1(A), r_2(B), w_2(B)$
swap $w_1(A)$ and $r_2(B)$:	$r_2(A), w_2(A), r_1(A), r_2(B), w_1(A), w_2(B)$
swap $w_1(A)$ and $w_2(B)$:	$r_2(A), w_2(A), r_1(A), r_2(B), w_2(B), w_1(A)$
swap $r_1(A)$ and $r_2(B)$:	$r_2(A), w_2(A), r_2(B), r_1(A), w_2(B), w_1(A)$
swap $r_1(A)$ and $w_2(B)$:	$r_2(A), w_2(A), r_2(B), w_1(A), w_2(B), r_1(A)$

Example: Good Schedule

- What's a non-serial example of a serializable schedule?
 - We could credit interest to A first, then withdraw the money, then B:
 - Schedule U : $r_2(A), w_2(A), r_1(A), w_1(A), r_2(B), w_2(B)$
 - Initial: $A = 200, B = 100$
 - Final: $A = 101, B = 100.50$
- Schedule U is conflict serializable to Schedule 2:

Schedule U:	$r_2(A), w_2(A), r_1(A), w_1(A), r_2(B), w_2(B)$
swap $w_1(A)$ and $r_2(B)$:	$r_2(A), w_2(A), r_1(A), r_2(B), w_1(A), w_2(B)$
swap $w_1(A)$ and $w_2(B)$:	$r_2(A), w_2(A), r_1(A), r_2(B), w_2(B), w_1(A)$
swap $r_1(A)$ and $r_2(B)$:	$r_2(A), w_2(A), r_2(B), r_1(A), w_2(B), w_1(A)$
swap $r_1(A)$ and $w_2(B)$:	$r_2(A), w_2(A), r_2(B), w_1(A), w_2(B), r_1(A)$

Example: Good Schedule

- What's a non-serial example of a serializable schedule?
 - We could credit interest to A first, then withdraw the money, then B:
 - Schedule U : $r_2(A), w_2(A), r_1(A), w_1(A), r_2(B), w_2(B)$
 - Initial: $A = 200, B = 100$
 - Final: $A = 101, B = 100.50$
- Schedule U is conflict serializable to Schedule 2:

Schedule U:	$r_2(A), w_2(A), r_1(A), w_1(A), r_2(B), w_2(B)$
swap $w_1(A)$ and $r_2(B)$:	$r_2(A), w_2(A), r_1(A), r_2(B), w_1(A), w_2(B)$
swap $w_1(A)$ and $w_2(B)$:	$r_2(A), w_2(A), r_1(A), r_2(B), w_2(B), w_1(A)$
swap $r_1(A)$ and $r_2(B)$:	$r_2(A), w_2(A), r_2(B), r_1(A), w_2(B), w_1(A)$
swap $r_1(A)$ and $w_2(B)$:	$r_2(A), w_2(A), r_2(B), w_2(B), r_1(A), w_1(A)$

The question that is it actually, I mean, does there exist a non-serial schedule which is serializable. Yes, it does. Suppose, we credit interest to A first then withdraw the money and then you credit interest to B. So, credit interest to A first is $r_2(A), w_2(A)$. So, it starts with 200, it becomes 201. Then you withdraw money, it becomes 101. Then you give interest to A. So, here it is a concurrent schedule which is, which gives the same effect as a serial schedule.

So, how do you get there? If I have this schedule how do I say that this is conflict serializable. I say look at every individual $w_1(A)$ and $r_2(B)$ these two they can be swapped, then $w_1(A)$ and $w_2(B)$ that can be swapped different, then $r_1(A), r_2(B)$ can be swapped and every time you are seeing what goes on here and finally $r_1(A)$ and $w_2(B)$. If you do that you get the whole of T2 first and then your T1 which actually gives you 101 rupees in the account of A.

So, this is your schedule 2 which is a serial schedule. So, you can see that any schedule, every schedule you may not be able to serialize, every concurrent schedule is not serializable, but there are concurrent schedules which were serializable and conflict serializability is actually trying to tell us that.

(Refer Slide Time: 19:17)



Serializability

Module 47

Partha Pratim Das

Objectives & Outline

Serializability

Conflicting operations

Conflict Serializability

Examples

Precedence graph

Deadlocks

Module Summary

- Are all serializable schedules conflict-serializable? No.
- Consider the following schedule for a set of three transactions.
 - $w_1(A), w_2(A), w_2(B), w_1(B), w_3(B)$
- We can perform no swaps to this:
 - The first two operations are both on A and at least one is a write
 - The second and third operations are by the same transaction;
 - The third and fourth are both on B at least one is a write; and
 - So are the fourth and fifth.
 - So this schedule is not conflict-equivalent to anything – and certain schedules.
- However, since nobody ever reads the values written by the $w_1(A), w_2(B)$ operations, the schedule has the same outcome as the serial schedule



Serializability

Module 47

Partha Pratim Das

Objectives & Outline

Serializability

Conflicting operations

Conflict Serializability

Examples

Precedence graph

Deadlocks

Module Summary

- Are all serializable schedules conflict-serializable? No.
- Consider the following schedule for a set of three transactions.
 - $w_1(A), w_2(A), w_2(B), w_1(B), w_3(B)$
- We can perform no swaps to this:
 - The first two operations are both on A and at least one is a write
 - The second and third operations are by the same transaction;
 - The third and fourth are both on B at least one is a write; and
 - So are the fourth and fifth.
 - So this schedule is not conflict-equivalent to anything – and certain schedules.
- However, since nobody ever reads the values written by the $w_1(A), w_2(B)$ operations, the schedule has the same outcome as the serial schedule



Serializability

Module 47

Partha Pratim Das

Objectives & Outline

Serializability

Conflicting operations

Conflict Serializability

Examples

Precedence Graph

Test

Module Summary

- Are all serializable schedules conflict-serializable? No.
- Consider the following schedule for a set of three transactions.
 - $w_1(A), w_2(A), w_2(B), w_1(B), w_3(B)$
- We can perform no swaps to this:
 - The first two operations are both on A and at least one is a write
 - The second and third operations are by the same transaction;
 - The third and fourth are both on B at least one is a write; and
 - So are the fourth and fifth.
 - So this schedule is not conflict-equivalent to anything – and certain schedules.
- However, since nobody ever reads the values written by the $w_1(A), w_1(B)$ operations, the schedule has the same outcome as the serial schedule



Serializability

Module 47

Partha Pratim Das

Objectives & Outline

Serializability

Conflicting operations

Conflict Serializability

Examples

Precedence Graph

Test

Module Summary

- Are all serializable schedules conflict-serializable? No.
- Consider the following schedule for a set of three transactions.
 - $w_1(A), w_2(A), w_2(B), w_1(B), w_3(B)$
- We can perform no swaps to this:
 - The first two operations are both on A and at least one is a write
 - The second and third operations are by the same transaction;
 - The third and fourth are both on B at least one is a write; and
 - So are the fourth and fifth.
 - So this schedule is not conflict-equivalent to anything – and certain schedules.
- However, since nobody ever reads the values written by the $w_1(A), w_1(B)$ operations, the schedule has the same outcome as the serial schedule

सिद्धिर्भवति कर्मजा

Serializability

Module 47
Partha Pratim Das
Objectives & Outline
Serializability
Conflicting Operations
Conflict Serializability
Examples
Precedence Graph
Deadlocks
Module Summary

- Are all serializable schedules conflict-serializable? No.
- Consider the following schedule for a set of three transactions.
 - $w_1(A), w_2(A), w_2(B), w_1(B), w_3(B)$
- We can perform no swaps to this:
 - The first two operations are both on A and at least one is a write
 - The second and third operations are by the same transaction;
 - The third and fourth are both on B at least one is a write; and
 - So are the fourth and fifth.
 - So this schedule is not conflict-equivalent to anything – and certain schedules.
- However, since nobody ever reads the values written by the $w_1(A), w_2(A)$ operations, the schedule has the same outcome as the serial schedule

Serializability

Module 47
Partha Pratim Das
Objectives & Outline
Serializability
Conflicting Operations
Conflict Serializability
Examples
Precedence Graph
Deadlocks
Module Summary

- Are all serializable schedules conflict-serializable? No.
- Consider the following schedule for a set of three transactions.
 - $w_1(A), w_2(A), w_2(B), w_1(B), w_3(B)$ ✓
- We can perform no swaps to this:
 - The first two operations are both on A and at least one is a write
 - The second and third operations are by the same transaction;
 - The third and fourth are both on B at least one is a write; and
 - So are the fourth and fifth.
 - So this schedule is not conflict-equivalent to anything – and certain schedules.
- However, since nobody ever reads the values written by the $w_1(A), w_2(A)$ operations, the schedule has the same outcome as the serial schedule

So, the final question is obviously if a schedule is conflict serializable, it is obviously serializable. There is no. But, so conflict serializability is a sufficient condition. But if, does there exist serializable schedules which are not conflict serializable? Yes, unfortunately there does. For example, look at this. There are three transactions $w_1(A)$ writing A , $w_2(A)$ writing A , transaction 2 writing B , transaction 1 writing B , transaction 3 writing B .

Now, you cannot perform any swap here. You cannot swap these two. They are working on the same data. You cannot swap these two because both are from the same transaction and they are in this order. So, we cannot change order of instruction within one transaction. So, you cannot swap them. You cannot swap these two, again the same data. You cannot swap these two

working on the same data. So, none of these, there is no swap possible. So, this schedule is not conflict equivalent to anything since you cannot do any swap at all and certainly not a serial schedule.

But the question is does a serial, is it equivalent to a serial schedule as such. Now, you can just, I mean, this is now plain logic. You can see that every time it is writing. There is no read. So, nobody has used that reading value. So, if I do the writes of T1, then the writes of T2 and then the write of T3, nothing changes, nothing will change, because I get the, what is the final effect that I will get in doing this. A will be, what has been written here, and B will be what has been written here.

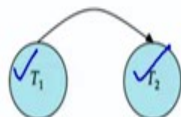
Here if you see after this A has not been written, so A will be what is written here and B will be what is written here. So, in this order which is a serial order of T1, T2, T3 you will get the same result as that concurrent schedule given here, but this concurrent schedule is not conflict equivalent to anything not to this serial schedule. So, it is serializable, but it is not conflict serializable.

So, the basic point is if something is conflict serializable then it is necessarily serializable, but not the other way around. Something, a schedule could be serializable but may not be conflict serializable, but such examples are rare.

(Refer Slide Time: 23:00)

Precedence Graph

- Consider some schedule of a set of transactions $T_1, T_2, \dots, T_n$
- Precedence Graph**
 - A direct graph where the vertices are the transactions (names)
- We draw an arc from T_i to T_j if the two transactions conflict, and T_i item on which the conflict arose earlier
- We may label the arc by the item that was accessed
- Example**



```
graph LR; T1((T1)) --> T2((T2))
```

Now, the question is how do you do this in an organized manner? So, how do you do this swap and thing in an automated fashion? So, what you do is you represent this in terms of a graph. This graph, the nodes of this graph are transactions and if it is, if there is a conflict from one transaction to the, between two instructions of two transactions, then you draw an arc from one transaction to the other, where T_i accesses the data item on which the conflict is existing.

You may label the arc by the item on which the conflict actually happened. And this is known as precedence graph. Why precedence graph, because this tells you the temporal order in which things have to happen. So, this is a representation of the schedule in terms of the transactions and their conflicts.

(Refer Slide Time: 24:16)

Testing for Conflict Serializability

Module 47
Partha Pratim Das
Objectives & Outline
Serializability
Conflicting operations
Conflict Serializability
Example
Precedence Graph
Tests
Module Summary

- A schedule is conflict serializable if and only if its precedence graph is acyclic
- Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph
 - (Better algorithms take order $n + e$ where e is the number of edges)
- If precedence graph is acyclic, the serializability order can be obtained by a topological sorting of the graph
 - That is, a linear order consistent with the partial order of the graph.

Diagram 1: A single node with a self-loop arrow.

Diagram 2: A vertical sequence of three nodes labeled T_1 , T_2 , and T_3 connected by downward arrows, representing a total order.



Module 47
Partha Pratim
Das

Database &
Databases

Serializability
locking
protocols

Conflict
Serializability
Concepts
Precedence graph

Tests

Module Summary

Testing for Conflict Serializability

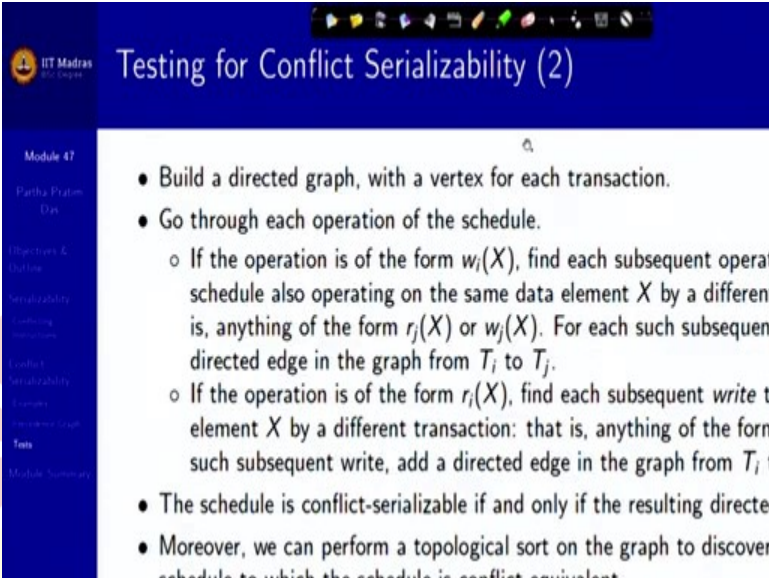
- A schedule is conflict serializable if and only if its precedence graph is acyclic
- Cycle-detection algorithms exist which take order n^2 time, where n is the number of vertices in the graph
 - (Better algorithms take order $n + e$ where e is the number of edges)
- If precedence graph is acyclic, the serializability order can be obtained by a topological sorting of the graph
 - That is, a linear order consistent with the partial order of the graph.



Now, we are making the algorithmic statement that a schedule is conflict serializable if and only if its precedence graph is acyclic. If the precedence graph does not have a cycle, then it is obviously conflict serializable. Why, because if it does not have a cycle, then I can always go in the in order, in the topological order of items of transactions and that will be valid and that gives you finally a serial. So, the topological sorting of the acyclic graph will give you the serial, the order in which the transactions will produce the effect. So, that is the basic model that we use.

If you are not clear about topological sorting how it is done, I would advise you to read up the algorithms book. It is a very standard breadth first technique to do that and it can happen in a, it can be done in, if there are n transactions, it can be done in order n square time, there is a better algorithm which does $n + e$ also, but even n square is fine, because usually the conflict graphs are not very large.

(Refer Slide Time: 25:49)

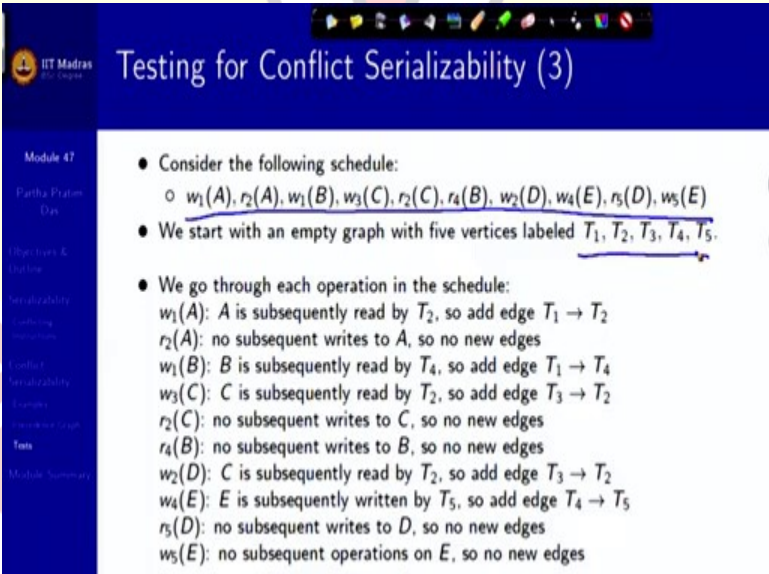


Testing for Conflict Serializability (2)

- Build a directed graph, with a vertex for each transaction.
- Go through each operation of the schedule.
 - If the operation is of the form $w_i(X)$, find each subsequent operation also operating on the same data element X by a different transaction, anything of the form $r_j(X)$ or $w_j(X)$. For each such subsequent directed edge in the graph from T_i to T_j .
 - If the operation is of the form $r_i(X)$, find each subsequent write operation on the same data element X by a different transaction: that is, anything of the form $w_j(X)$. For each such subsequent write, add a directed edge in the graph from T_i to T_j .
- The schedule is conflict-serializable if and only if the resulting directed graph has no cycle.
- Moreover, we can perform a topological sort on the graph to discover a serializable schedule to which the schedule is conflict equivalent.

So, this is the process by which you do that, you read this, understand, but I will just walk you through an example for better understanding.

(Refer Slide Time: 25:55)



Testing for Conflict Serializability (3)

- Consider the following schedule:
 $w_1(A), r_2(A), w_1(B), w_3(C), r_2(C), r_4(B), w_2(D), w_4(E), r_5(D), w_5(E)$
- We start with an empty graph with five vertices labeled T_1, T_2, T_3, T_4, T_5 .
- We go through each operation in the schedule:
 - $w_1(A)$: A is subsequently read by T_2 , so add edge $T_1 \rightarrow T_2$
 - $r_2(A)$: no subsequent writes to A, so no new edges
 - $w_1(B)$: B is subsequently read by T_4 , so add edge $T_1 \rightarrow T_4$
 - $w_3(C)$: C is subsequently read by T_2 , so add edge $T_3 \rightarrow T_2$
 - $r_2(C)$: no subsequent writes to C, so no new edges
 - $r_4(B)$: no subsequent writes to B, so no new edges
 - $w_2(D)$: D is subsequently read by T_5 , so add edge $T_2 \rightarrow T_5$
 - $w_4(E)$: E is subsequently written by T_5 , so add edge $T_4 \rightarrow T_5$
 - $r_5(D)$: no subsequent writes to D, so no new edges
 - $w_5(E)$: no subsequent operations on E, so no new edges

Testing for Conflict Serializability (3)

Module 47
Partha Pratim Das

- Consider the following schedule:
 - $w_1(A), r_2(A), w_1(B), w_3(C), r_2(C), r_4(B), w_2(D), w_4(E), r_5(D), w_5(E)$
- We start with an empty graph with five vertices labeled T_1, T_2, T_3, T_4, T_5 .
- We go through each operation in the schedule:
 - $w_1(A)$: A is subsequently read by T_2 , so add edge $T_1 \rightarrow T_2$
 - $r_2(A)$: no subsequent writes to A, so no new edges
 - $w_1(B)$: B is subsequently read by T_4 , so add edge $T_1 \rightarrow T_4$
 - $w_3(C)$: C is subsequently read by T_2 , so add edge $T_3 \rightarrow T_2$
 - $r_2(C)$: no subsequent writes to C, so no new edges
 - $r_4(B)$: no subsequent writes to B, so no new edges
 - $w_2(D)$: D is subsequently read by T_5 , so add edge $T_2 \rightarrow T_5$
 - $w_4(E)$: E is subsequently written by T_5 , so add edge $T_4 \rightarrow T_5$
 - $r_5(D)$: no subsequent writes to D, so no new edges
 - $w_5(E)$: no subsequent operations on E, so no new edges

Testing for Conflict Serializability (3)

Module 47
Partha Pratim Das

- Consider the following schedule:
 - $w_1(A), r_2(A), w_1(B), w_3(C), r_2(C), r_4(B), w_2(D), w_4(E), r_5(D), w_5(E)$
- We start with an empty graph with five vertices labeled T_1, T_2, T_3, T_4, T_5 .
- We go through each operation in the schedule:
 - $w_1(A)$: A is subsequently read by T_2 , so add edge $T_1 \rightarrow T_2$
 - $r_2(A)$: no subsequent writes to A, so no new edges
 - $w_1(B)$: B is subsequently read by T_4 , so add edge $T_1 \rightarrow T_4$
 - $w_3(C)$: C is subsequently read by T_2 , so add edge $T_3 \rightarrow T_2$
 - $r_2(C)$: no subsequent writes to C, so no new edges
 - $r_4(B)$: no subsequent writes to B, so no new edges
 - $w_2(D)$: D is subsequently read by T_5 , so add edge $T_2 \rightarrow T_5$
 - $w_4(E)$: E is subsequently written by T_5 , so add edge $T_4 \rightarrow T_5$
 - $r_5(D)$: no subsequent writes to D, so no new edges
 - $w_5(E)$: no subsequent operations on E, so no new edges

$T_3 - T_1$
 $T_1 - T_2$

So, this is a schedule given. So, we have data item A, B, C, D, E, five data items and five transactions. So, we have start with an empty graph with vertices. Now, you have $w_1(A)$ being written and it is being read by transaction 2, T_2 . So, we will have an edge T_1 to, so whatever T_1 writes is what T_2 reads. So, T_1 has to write first. So, that gives you this edge. We go to r_2 . There is no further reference of A, so this has no effect.

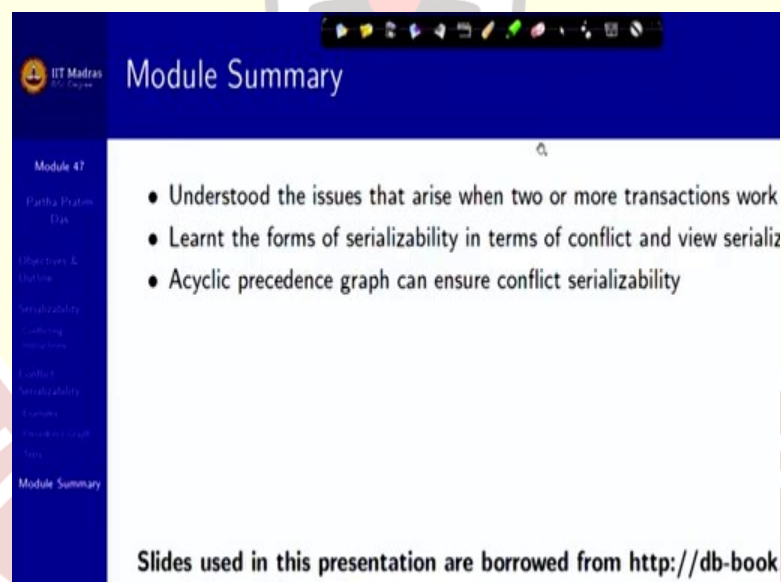
Then we have $w_1(B)$. $w_1(B)$ is read here. So, whatever transaction 1 writes in B, transaction 4 reads. So, transaction 1 has to precede transaction 4 in terms of this. $w_3(C)$ transaction 3 writes and transaction 2 reads. So, 3 to 2. You have this. Then there is nothing else on B. There is nothing else on C. Now, $w_2(D)$ is read by transaction 5. So, you have 2 to 5. Then finally, you

have $w_4(E)$ what is written by E in transaction 4 is overwritten by transaction 5 and you are done.

So, what you have got that these transactions, this schedule has a precedence graph like this. And you can clearly see that this is an acyclic graph. Since this is an acyclic graph, we can sort it by topological sort. For example, a one topological sort could be T3, T1, then T2, T4, T5. There could be different sorts. For example, T1, T3, T4, T2, T5 is also another. So, actually this is equivalent to a number of different serial schedules, but we just need one.

Even we do not need to produce that, but the fact that this precedence graph is acyclic tells me that this schedule is conflict serializable. And if you actually follow these precedences, you will be able to generate that sequence also. In the previous slide actually this process that we executed here is written in words which is a little bit more difficult to understand. So, I focused on the example. You can now go back and read it.

(Refer Slide Time: 29:37)



Module Summary

- Understood the issues that arise when two or more transactions work
- Learnt the forms of serializability in terms of conflict and view serializ
- Acyclic precedence graph can ensure conflict serializability

Slides used in this presentation are borrowed from <http://db-book>

So, this brings us to the end of this module. We have discussed about the concurrency issues of two or more transactions and the need for serializability and learned that there are two types of serializability conflict serializability and view serializability and we have specifically discussed about conflict serializability with the conflict in instructions and the properties that a schedule has to have to be conflict serializable. We will continue on this in the next module. Thank you very much for your attention. See you in the next module.